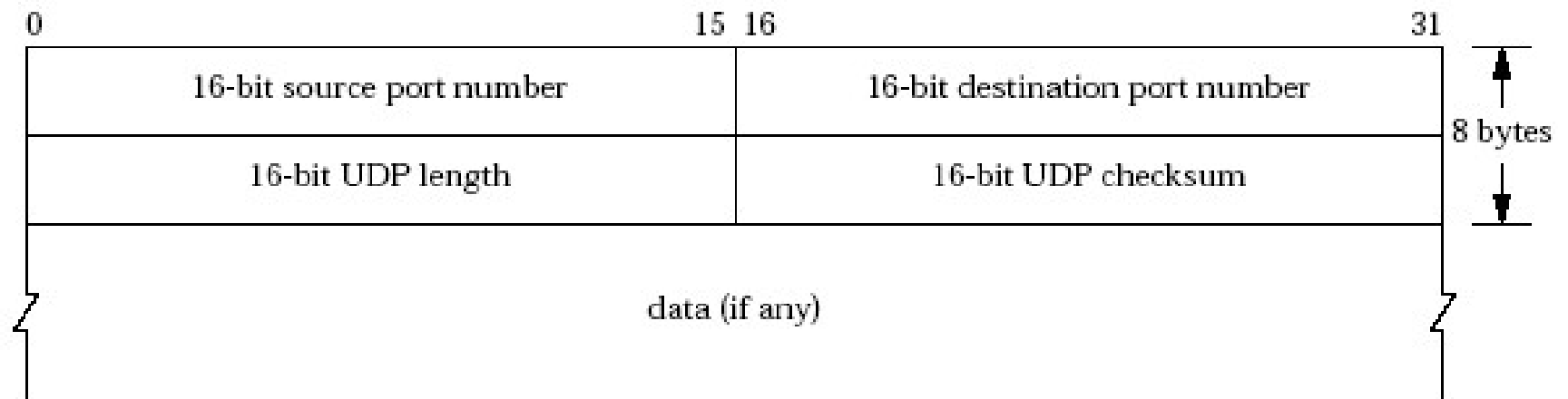

UDP Socket

UDP

- ❑ Dịch vụ không kết nối
- ❑ Không cung cấp: thiết lập kết nối, tin cậy, điều khiển dòng, thời gian, ...
- ❑ Tại sao sử dụng UDP?



UDP

❑ Nhược điểm

- Các thông điệp có thể được nhận theo bất kỳ thứ tự nào.
- Không có gì đảm bảo là các gói tin sẽ đến đích

❑ Ưu điểm:

- UDP một giao thức có tốc độ truyền tin nhanh
- Truyền tin unicast, broadcast và multicast.

❑ Kiểu truyền

- Một thông điệp unicast được gửi từ nút này tới nút khác.
- Truyền tin broadcast nghĩa là một thông điệp có thể được gửi tới tất cả các nút trong một mạng.
- Multicast cho phép các thông điệp được truyền tới một nhóm các nút được lựa chọn.

Lập trình với Socket UDP

UDP: không kết nối giữa client và server

- ❑ Không bắt tay
- ❑ Phía gửi sẽ gán địa chỉ IP và Port của đích vào mỗi đoạn dữ liệu.
- ❑ OS sẽ đính kèm IP và Port của máy gửi vào mỗi đoạn.
- ❑ Server loại bỏ phần IP và port của máy gửi và nhận đoạn dữ liệu.

Góc độ ứng dụng

UDP cung cấp cách thức gửi không đảm bảo một nhóm byte ("datagrams") giữa client và server

Ví dụ

□ Client:

- Người gõ một đoạn văn bản.
- Client gửi dòng văn bản tới server.

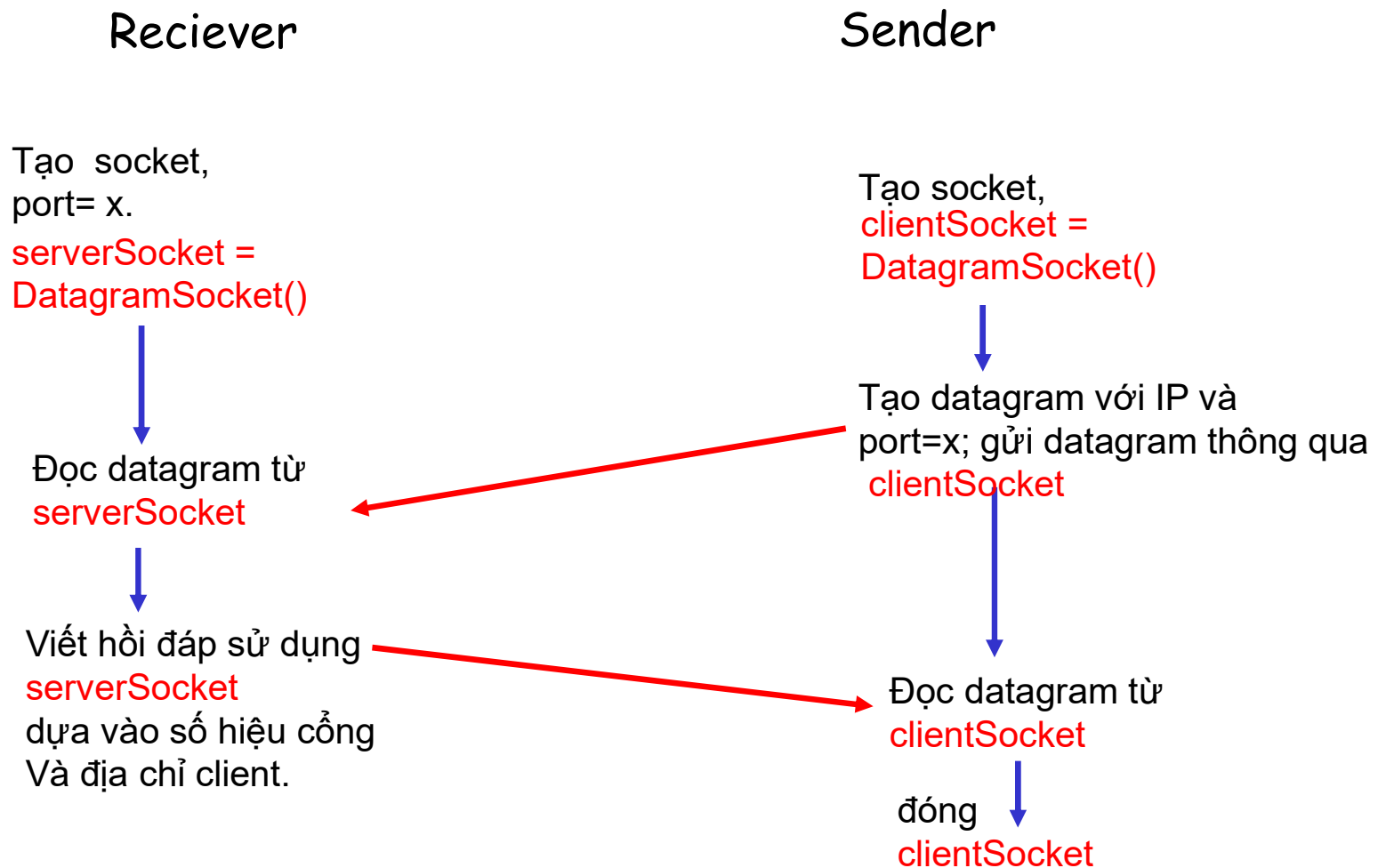
□ Server:

- Server nhận dòng văn bản.
- Đổi hoa tất cả các ký tự.
- Gửi đoạn văn bản đã sửa đổi về cho client.

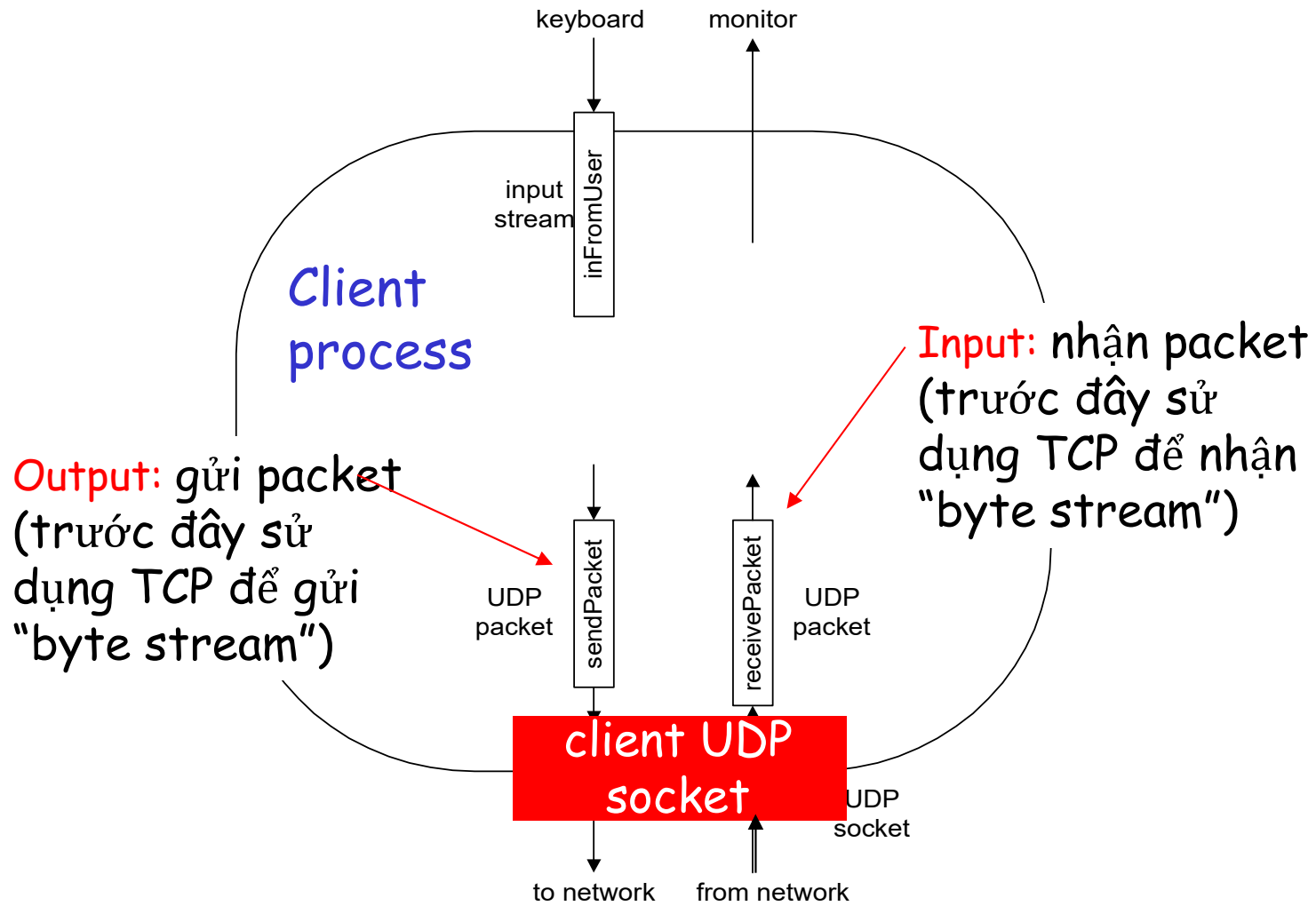
□ Client:

- Nhận dòng văn bản.
- Hiển thị.

Tương tác giữa socket sender/reciever



Ví dụ: Java client (UDP)



Ví dụ: Java client (UDP)

```
import java.io.*;  
import java.net.*;
```

```
class UDPClient {  
    public static void main(String args[]) throws Exception  
    {
```

Tạo
Luồng nhập

```
        BufferedReader inFromUser =  
            new BufferedReader(new InputStreamReader(System.in));
```

Tạo socket client

```
        DatagramSocket clientSocket = new DatagramSocket();
```

Dịch tên miền
hostname sang
địa chỉ IP
sử dụng DNS

```
        InetAddress IPAddress = InetAddress.getByName("localhost");
```

```
        byte[] sendData = new byte[1024];  
        byte[] receiveData = new byte[1024];
```

```
        String sentence = inFromUser.readLine();  
        sendData = sentence.getBytes();
```


Ví dụ: Java client UDP

Tạo datagram với dữ

liệu, độ dài dữ liệu,
địa chỉ IP, số hiệu
cổng

```
DatagramPacket sendPacket =  
    new DatagramPacket(sendData, sendData.length, IPAddress, 9876);
```

Gửi datagram
Tới server

```
clientSocket.send(sendPacket);
```

```
DatagramPacket receivePacket =  
    new DatagramPacket(receiveData, receiveData.length);
```

Đọc datagram
từ server

```
clientSocket.receive(receivePacket);
```

```
String modifiedSentence =  
    new String(receivePacket.getData());
```

```
System.out.println("FROM SERVER:" + modifiedSentence);  
clientSocket.close();  
}
```

```
}
```

Ví dụ: Java server UDP

```
import java.io.*;  
import java.net.*;
```

```
class UDPServer {  
    public static void main(String args[]) throws Exception  
    {
```

Tạo datagram socket
Lắng nghe tại
port 9876



```
DatagramSocket serverSocket = new DatagramSocket(9876);
```

```
byte[] receiveData = new byte[1024];  
byte[] sendData = new byte[1024];
```

```
while(true)  
{
```

Cấp phát bộ nhớ
để nhận gói dữ liệu



```
DatagramPacket receivePacket =  
    new DatagramPacket(receiveData, receiveData.length);
```

Nhận gói
dữ liệu



```
serverSocket.receive(receivePacket);
```

Ví dụ: Java server UDP

```
String sentence = new String(receivePacket.getData());
```

Lấy địa chỉ IP,
port của gói gửi

```
InetAddress IPAddress = receivePacket.getAddress();
```

```
int port = receivePacket.getPort();
```

```
String capitalizedSentence = sentence.toUpperCase();
```

```
sendData = capitalizedSentence.getBytes();
```

Tạo datagram
để gửi tới client

```
DatagramPacket sendPacket =  
    new DatagramPacket(sendData, sendData.length, IPAddress,  
                        port);
```

Viết
Datagram
ra socket

```
serverSocket.send(sendPacket);
```

```
}  
}  
}
```

Kết thúc lặp
và đợi gói dữ liệu khác

Quan sát và đặt câu hỏi về UDP

- ❑ Cả client server đều sử dụng DatagramSocket
- ❑ Đại chỉ IP và port của máy đích được gửi kèm cho mỗi đoạn dữ liệu.
- ❑ Client gửi đoạn dữ liệu tới server mà không cần địa chỉ IP và port ?
- ❑ Nhiều client có thể dùng một server?

DatagramPacket

- ❑ Kích thước tối đa một gói dữ liệu là 65,507 byte
- ❑ Phương thức khởi tạo
 - `public DatagramPacket(byte[] buffer, int length)`
 - `public DatagramPacket(byte[] buffer, int offset, int length)`
- ❑ Các phương thức khởi tạo khác
 - `public DatagramPacket(byte[] data, int length, InetAddress destination, int port)`
 - `public DatagramPacket(byte[] data, int offset, int length, InetAddress destination, int port)`
 - `public DatagramPacket(byte[] data, int length, SocketAddress destination, int port)`
 - `public DatagramPacket(byte[] data, int offset, int length, SocketAddress destination, int port)`

Ví dụ

```
String s = "On essaie...";  
byte[] data = s.getBytes("ASCII");  
  
try {  
    InetAddress ia = InetAddress.getByName("it.hutech.edu.vn");  
    int port = 7;  
    DatagramPacket dp = new DatagramPacket(data,  
        data.length, ia, port);  
}  
catch (IOException ex)  
{  
}
```

Các phương thức

❑ Thao tác với địa chỉ

- public InetAddress getAddress()
- public int getPort()
- public SocketAddress
getSocketAddress()
- public void setAddress(InetAddress remote)
- public void setPort(int port)
- public void setAddress(SocketAddress remote)

Các phương thức

❑ Thao tác với dữ liệu

- `public byte[] getData( )`
- `public int getLength( )`
- `public int getOffset( )`
- `public void setData(byte[] data)`
- `public void setData(byte[] data, int offset, int length )`
- `public void setLength(int length)`

Ví du

```
import java.net.*;
public class DatagramExample {
    public static void main(String[] args) {
        String s = "Essayons.";
        byte[] data = s.getBytes( );
        try {
            InetAddress ia = InetAddress.getByName("www.liafa.jussieu.fr");
            int port =7;
            DatagramPacket dp;
            dp = new DatagramPacket(data, data.length, ia, port);
            System.out.println(" Un packet pour" + dp.getAddress( ));
            System.out.println(" port " + dp.getPort( ));
            System.out.println("il y a " + dp.getLength( ));
            System.out.println(" bytes dans le packet");
            s=new String(dp.getData( ), dp.getOffset( ), dp.getLength( ))
            System.out.println(s);
        }
        catch (UnknownHostException e) {
            System.err.println(e);
        }
    }
}
```

DatagramSocket

❑ Phương thức khởi tạo

- `public DatagramSocket( ) throws SocketException`
- `public DatagramSocket(int port) throws SocketException`
- `public DatagramSocket(int port, InetAddress interface) throws SocketException`
- `public DatagramSocket(SocketAddress interface) throws SocketException`
- `(protected DatagramSocket(DatagramSocketImpl impl) throws SocketException)`

Ví dụ

```
java.net.*;
public class UDPPortScanner {
    public static void main(String[] args) {

        for (int port = 1024; port <= 65535; port++) {
            try {
                // kiểm tra có port nào không được dùng ko?
                //nếu có, ngoại lệ sẽ được ném ra
                DatagramSocket server = new DatagramSocket(port);
                server.close( );
            }
            catch (SocketException ex) {
                System.out.println("Port:" + port + " đã được dùng");
            } // end try
        } // end for
    }
}
```

Gửi và nhận

- ❑ `public void send(DatagramPacket dp) throws IOException`
- ❑ `public void receive(DatagramPacket dp) throws IOException`

Ví dụ: Echo

- ❑ UDPServer
 - UDPEchoServer
- ❑ UDPEchoClient
 - SenderThread
 - ReceiverThread

Echo: UDPServer

```
import java.net.*;
import java.io.*;
public abstract class UDPServer extends
    Thread {
    private int bufferSize;
    protected DatagramSocket sock;
    public UDPServer(int port, int bufferSize)
        throws SocketException {
        this.bufferSize = bufferSize;
        this.sock = new DatagramSocket(port);
    }
    public UDPServer(int port) throws
        SocketException {
        this(port, 8192);
    }
}
```

```
public void run() {
    byte[] buffer= new byte[bufferSize];
    while (true) {
        DatagramPacket incom;
        incom = new DatagramPacket(buffer,
            buffer.length);
        try {
            sock.receive(incom);
            this.respond(incom);
        } catch (IOException e) {
            System.err.println(e);
        }
    } // end while
}
public abstract void
    respond(DatagramPacket request);
}
```

UDPEchoServer

```
public class UDPEchoServer extends UDP Server {
    public final static int DEFAULT_PORT = 2222;
    public UDPEchoServer() throws SocketException {
        super(DEFAULT_PORT);
    }
    public void respond(DatagramPacket packet) {
        try {
            byte[] data = new byte[packet.getLength()];
            System.arraycopy(packet.getData(), 0, data, 0, packet.getLength());
            try {
                String s = new String(data, "8859_1");
                System.out.println(packet.getAddress() + " port "
                    + packet.getPort() + " goi: " + s);
            } catch (java.io.UnsupportedEncodingException ex) {}
            DatagramPacket outgoing = new DatagramPacket(packet.getData(),
                packet.getLength(), packet.getAddress(), packet.getPort());
            sock.send(outgoing);
        } catch (IOException ex) {
            System.err.println(ex);
        }
    }
}
```

Client: UDPEchoClient

```
public class UDPEchoClient {  
    public static void lancer(String hostname, int port) {  
        try {  
            InetAddress ia = InetAddress.getByName(hostname);  
            SenderThread sender = new SenderThread(ia, port);  
            sender.start();  
            Thread receiver = new ReceiverThread(sender.getSocket());  
            receiver.start();  
        }  
        catch (UnknownHostException ex) {  
            System.err.println(ex);  
        }  
        catch (SocketException ex) {  
            System.err.println(ex);  
        }  
    } // end  
}
```


ReceiverThread

```
class ReceiverThread extends Thread {
    DatagramSocket socket;
    private boolean stopped = false;
    public ReceiverThread(DatagramSocket ds) throws SocketException {
        this.socket = ds;
    }
    public void halt() {      this.stopped = true;  }
    public DatagramSocket getSocket(){      return socket;  }
    public void run() {
        byte[] buffer = new byte[65507];
        while (true) {
            if (stopped) return;
            DatagramPacket dp = new DatagramPacket(buffer, buffer.length);
            try {
                socket.receive(dp);
                String s = new String(dp.getData(), 0, dp.getLength());
                System.out.println(s);
                Thread.yield();
            } catch (IOException ex) {System.err.println(ex); }
        }
    }
}
```

SenderThread

```
public class SenderThread extends Thread {
    private InetAddress server;
    private DatagramSocket socket;
    private boolean stopped = false;
    private int port;
    public SenderThread(InetAddress address, int port)
        throws SocketException {
        this.server = address;
        this.port = port;
        this.socket = new DatagramSocket();
        this.socket.connect(server, port);
    }
    public void halt() {
        this.stopped = true;
    }
    //...
```

SenderThread

```
public DatagramSocket getSocket() {
    return this.socket;
}
public void run() {
    try {
        BufferedReader userInput = new BufferedReader(new
        InputStreamReader(System.in));
        while (true) {
            if (stopped) return;
            String theLine = userInput.readLine();
            if (theLine.equals(".")) break;
            byte[] data = theLine.getBytes();
            DatagramPacket output;
            output = new DatagramPacket(data, data.length, server, port);
            socket.send(output);
            Thread.yield();
        }
    } // end try
    catch (IOException ex) {System.err.println(ex); }
} // end run
}
```

Các phương thức khác

- ❑ `public void close( )`
- ❑ `public int getLocalPort( )`
- ❑ `public InetAddress getLocalAddress( )`
- ❑ `public SocketAddress getLocalSocketAddress( )`
- ❑ `public void connect(InetAddress host, int port)`
- ❑ `public void disconnect( )`
- ❑ `public void disconnect( )`
- ❑ `public int getPort( )`
- ❑ `public InetAddress getInetAddress( )`
- ❑ `public InetAddress getRemoteSocketAddress( )`

Options

- ❑ `SO_TIMEOUT`
 - `public synchronized void setSoTimeout(int timeout) throws SocketException`
 - `public synchronized int getSoTimeout( ) throws IOException`
- ❑ `SO_RCVBUF`
 - `public void setReceiveBufferSize(int size) throws SocketException`
 - `public int getReceiveBufferSize( ) throws SocketException`
- ❑ `SO_SNDBUF`
 - `public void setSendBufferSize(int size) throws SocketException`
 - `int getSendBufferSize( ) throws SocketException`
- ❑ `SO_REUSEADDR`
 - `public void setReuseAddress(boolean on) throws SocketException`
 - `boolean getReuseAddress( ) throws SocketException`
- ❑ `SO_BROADCAST`
 - `public void setBroadcast(boolean on) throws SocketException`
 - `public boolean getBroadcast( ) throws SocketException`

Client

```
import java.io.*; import java.net.*; class udpclientchat {
public static void main(String[] args) throws Exception {
    byte []sendData; byte []buffer = new byte[1024]; boolean ktFinish = false; BufferedReader in = new
        BufferedReader(new InputStreamReader(System.in)); //2.UDP client request connecting
        System.out.print("\n2.UDP client start and connecting...");
        DatagramSocket socket = new DatagramSocket(); InetAddress ipServer =
        InetAddress.getByName("localhost");
        int port = 2812; //3.Send status connected server
        String stSend = "UDP Server -CLient connected."; System.out.print("\n3."+stSend); sendData =
        stSend.getBytes();
        DatagramPacket sendPacket = new DatagramPacket(sendData,sendData.length,ipServer,port);
        socket.send(sendPacket); //Begin chat
        while(ktFinish!=true) {
            DatagramPacket receivePacket = new DatagramPacket(buffer,buffer.length);
            socket.receive(receivePacket);
            String stReceive = new String(receivePacket.getData(),0,receivePacket.getLength());
            System.out.println("\nServer: " + stReceive); //UDP Client data to UDP Server
            System.out.print("\nCLient:"); stSend= in.readLine();
            sendData=stSend.getBytes(); sendPacket = new
            DatagramPacket(sendData,sendData.length,ipServer,port);
            socket.send(sendPacket); //finish
            if (stSend.equals("end.")||stReceive.equals("end.")) { ktFinish = true; }
        }
        socket.close();
    }}
}
```

Chat 2

```
import java.io.*; import java.net.*; class UDPClientChat {  
    public static void main(String[] args) throws Exception {  
        BufferedReader in = new BufferedReader (new InputStreamReader (System.in));  
        InetAddress ipServer = InetAddress.getByName("localhost"); //serverName  
        byte buffer[] = new byte[1024];  
        int portServer = 8888;
```

Chat 2

```
try {
    DatagramSocket socket = new DatagramSocket();
    DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
    boolean ketthuc = false;
    String stgui="", stnhan="";
    do { //Gui
        byte sendData[];
        System.out.print("Client: ");
        stgui = in.readLine();
        sendData = stgui.getBytes();
        DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, ipServer, portServer);
        socket.send(sendPacket);
        if (!stgui.equalsIgnoreCase("bye")) { //Nhan
            socket.receive(packet);
            stnhan = new String(packet.getData(),0,packet.getLength());
            System.out.println("Server: " +stnhan);
        }
        if (stgui.equalsIgnoreCase("bye") || stnhan.equalsIgnoreCase("bye")) { ketthuc = true; }
    } while (!ketthuc); //ketthuc=false
    socket.close();
} catch (UnknownHostException e) { e.printStackTrace(); } }
```


Server-Chat2

```
import java.io.*; import java.net.*;
class UDPServerChat {
public static void main(String[] args) throws Exception {
    BufferedReader in = new BufferedReader (new InputStreamReader (System.in));
    byte buffer[] = new byte[1024]; int port = 8888;
    try {    DatagramSocket socket = new DatagramSocket(port);
        System.out.println("May chu doi ket noi o cong 8888...");
        DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
        boolean ketthuc = false; String stgui="", stnhan="";
        do { //Nhan
            socket.receive(packet);
            stnhan = new String(packet.getData(),0,packet.getLength());
            System.out.println("Client: " +stnhan);
            if (!stnhan.equalsIgnoreCase("bye")) { //Gui
                InetAddress ipClient = packet.getAddress();
                int portClient = packet.getPort(); byte sendData[];
                System.out.print("Server: "); stgui = in.readLine();
                sendData = stgui.getBytes();
                DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, ipClient, portClient);
                socket.send(sendPacket);
            }
        } while (!ketthuc); //ketthuc=false socket.close(); }
    catch (UnknownHostException e) { e.printStackTrace(); } } }
```

Bài tập

1. Viết chương trình Client-Server sử dụng giao thức **UDP** cho phép:

- Client : gửi chuỗi chữ thường lên Server
- Server: Nhận chuỗi và chuyển thành HOA, trả về Client

2. Viết chương trình Client-Server sử dụng giao thức **UDP** cho phép:

- Client : gửi 2 số và phép tính (+ - * / %) lên Server
- Server: Nhận và thực hiện tính toán và trả về Client

BÀI TẬP: GIAO THỨC UDP

BÀI TẬP

Viết chương trình giao tiếp Client – Server sử dụng giao thức UDP

Tìm số fibo thứ n.